Now let's make use of `getCounter()`. To assign the returned value by reference, we need another `&` (told you it was cumbersome), like so:

```
$countOne = &getCounter();
$countTwo = &getCounter();
$countOne++;
var_dump($countTwo);
$countTwo++;
var_dump($countOne);
```

```
Output:
int(1)
int(2)
```

`$countOne` and `$countTwo` are both referencing the locally scoped `$i` variable inside `getCounter()`. When we increment one, the new value is accessible from the other.

Declaring Types

It is good practice to define the data types that functions accept and return. This provides information to other programmers about how to use the function in an easily digested form (note that "other programmers" includes future you). A single statement (the beginning of the function declaration) can tell you at a glance how the function expects to be called and what it will evaluate to in expressions.

PHP 8 allows you to declare expected types for incoming arguments and returned data. Since PHP 7.0, every base data type is supported in function declarations with one exception: the `resource` type is not permitted (`resource` is a bit of a special case, as you might recall). Otherwise, you are free to use `string`, `int`, `float`, `bool`, `array`, and so on. Expanding beyond this, the type declaration system offers additional *pseudo-types*, such as constructs for declaring sets of types and types for defining function behavior. Pseudo-types only exist for use in declarations; you cannot cast data to these types.

Remember that types will be coerced unless PHP has been configured for strict behavior with a `declare(strict_types=1)` declaration, so a string that can be cast to an integer will be a legal value for a function that requires integer data (with all the caveats that entails).